

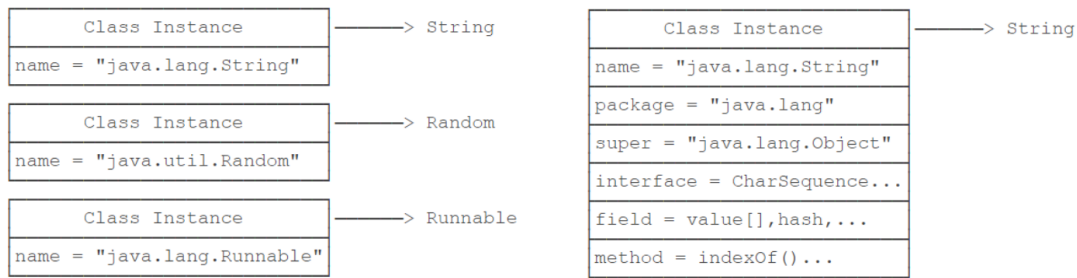
Lecture 10

• Reflection

- 反射是 Java 的一个特性，位于 `java.lang.reflect` 包中
- 反射用于在运行时检查或修改方法、类、接口的行为
 - 检查一个类的所有字段和方法
 - 调用对象的方法
 - 从另一个类访问私有字段

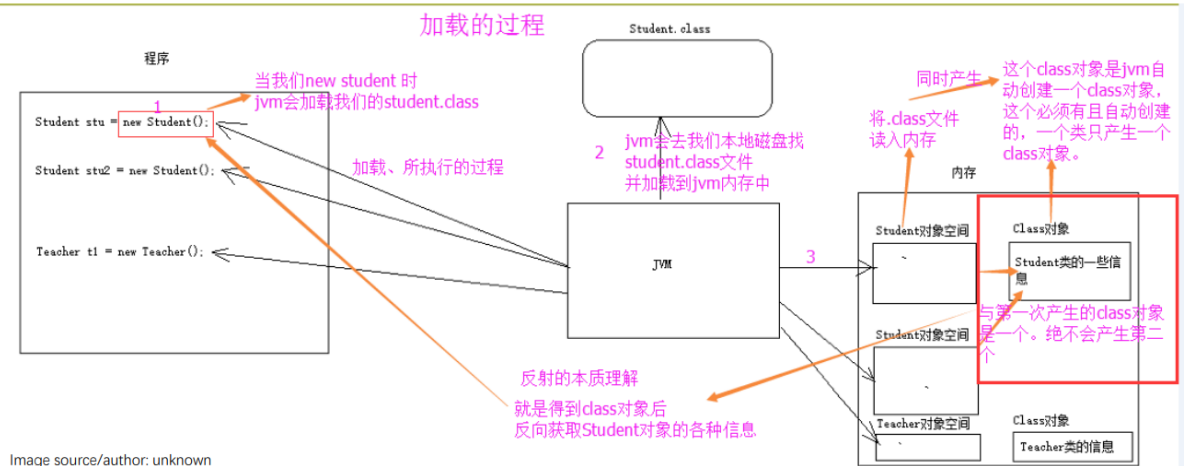
```
Object x = . . .;
if (x instanceof Shape)
{
    Shape s = (Shape) x;
    g2.draw(s);
}
```

- `x` 可能来自用户输入、JSON 文件、服务器响应、序列化数据等；我们不知道它的准确类型
- 即便使用 `instanceof`，我们仍然不知道 `x` 的精确类型（它可能是某个子类，例如 `Rectangle`）
- 如果需要针对不同类型的 `x` 执行不同操作，是否要写很多个 `instanceof` 判断块？
- 示例说明：判断 `x` 是否是 `Shape` 类型，若是则转换并绘制（示例代码省略）
- 确定实际类型：
 - 如果你有任何对象引用，可以使用 `getClass()` 方法找到该引用所指对象的实际类型
 - `getClass()` 方法返回一个 `Class` 类型的对象，用于描述该对象所属的类 `Class c = x.getClass();`
 - 示例：使用 `Class` 实例可以获取大量关于该类的信息（如名称、字段、构造函数、方法等）
- `Class` 类
 - JVM 为每种数据类型（包括原始类型、类和接口）创建一个对应的 `Class` 实例
 - 每个 `Class` 实例都保存对应类的详细信息，例如：类名、包名、父类、所实现的接口、字段信息、方法信息等



- 通过 Class 实例可以获得大量关于该类的元数据，用于反射操作
- JVM 加载过程
 - JVM 加载类（.class 文件）时会自动构建对应的 Class 实例
 - 每当 JVM 加载某个类/类型（例如 String）时，都会为其创建一个 Class 类型的实例
 - 注意：Class 类没有公共构造函数，因此我们不能手动创建 Class 对象，只有 JVM 能创建它们
 - （补充说明：Class 的构造器由 JVM 内部使用，普通代码无法调用）

JVM Loading Process



- 当执行 new Student() 时，JVM 会去本地磁盘查找并加载 student.class 文件到内存
- JVM 为该类型自动创建一个 Class 对象，并将 .class 文件读入内存
- Class 对象保存该类的一些信息（类名、字段、方法等）
- 不管创建多少个该类的实例（如 stu、stu2），对应的 Class 对象只会在第一次加载时由 JVM 创建一次，永远只有一个
- 获取 Class 对象的方式：
 - 方法一（已知类时）：使用 .class 语法获取 Class 对象


```
Class cls1 = String.class;
```
 - 方法二（运行时按名字获取）：使用 Class.forName(完整包名) 根据类的全限定名获取 Class 对象


```
Class cls2 = Class.forName("java.lang.String");
```

- 方法三（已有实例时）：对对象调用 `getClass()` 方法获取其对应的 Class 对象

```
String s = "Hello";
Class cls3 = s.getClass();
```

- 可以使用 `==` 运算符来测试两个 Class 实例是否表示相同的类型（即比较它们是否为同一个 Class 对象）
- 获取 Class Names
 - 使用对象的 Class 实例并调用 `getName()` 可以得到该对象的精确类名

```
String s = "Hello";
System.out.println(s.getClass().getName());
```

`java.lang.String`

- 数组类型名的特殊编码
 - 出于历史原因，`getName()` 对数组类型返回特殊的编码字符串（不是常见的点分包名）
 - 示例：`double[].class.getName()` → `"[D"`，`String[][] .class.getName()` → `"[[Ljava.lang.String;"`
 - 规则概述：每个维度一个前缀 `[]`；原始类型用单字母编码（B byte、C char、D double、F float、I int、J long、S short、Z boolean）；引用类型用 `"L全限定类名;"` 表示
- 获取类字段（Field）的方式
 - 常用方法：
 - `Field getField(name)`：按名字获取公共字段（包括继承来的 public）
 - `Field getDeclaredField(name)`：按名字获取当前类声明的字段（不包括继承的字段）
 - `Field[] getFields()`：获取所有公共字段（包括继承来的 public）
 - `Field[] getDeclaredFields()`：获取当前类声明的所有字段（包括 public、protected、默认、private，但不包括继承字段）
- Field 类（字段对象）的用途：
 - Field 提供关于单个字段的元信息，并允许在运行时动态读取/写入该字段的值（受访问权限限制）

```
Field f = String.class.getDeclaredField( name: "value");
System.out.println(f.getName()); // value
System.out.println(f.getType()); // [B

int m = f.getModifiers();
System.out.println(Modifier.isFinal(m)); // true
System.out.println(Modifier.isPrivate(m)); // true
```

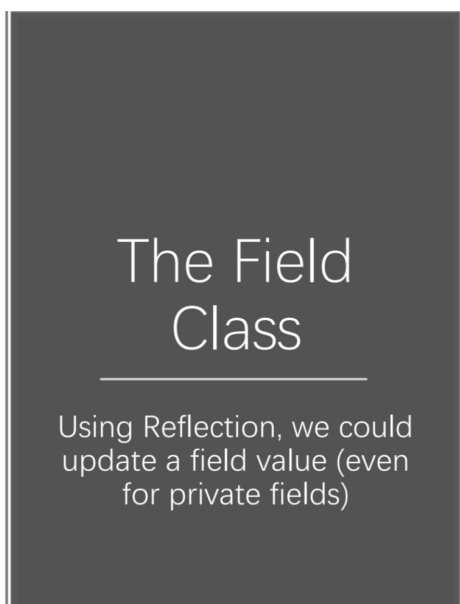
```
public final class String
implements java.io.Serializable, Comparable<String>, CharSequence,
    Constable, ConstantDesc {

    The value is used for character storage.
    Implementation This field is trusted by the VM, and is a subject to constant folding i
    Note: is constant. Overwriting this field after construction will cause probl
    Additionally, it is marked with Stable to trust the contents of the ar
    facility in JDK provides this functionality (yet). Stable is safe here, b
    never null.

    @Stable
    private final byte[] value;
```

- `int m = f.getModifiers();` 返回表示修饰符的整数，使用 `Modifier` 类解码（如 `Modifier.isFinal(m)`、`Modifier.isPrivate(m)`）

- getModifiers() 返回字段的 Java 修饰符表示，应使用 Modifier 辅助方法解析具体修饰符



```
BankAccount bc = new BankAccount();
System.out.println(bc.getBalance()); // 0.0

Class clz = bc.getClass();
// get the private field 获取该类声明的私有字段Field f
Field f = clz.getDeclaredField("balance");
// make the private field accessible
f.setAccessible(true); 将私有字段设为可访问的
// set the balance
f.set(bc, 1000);
System.out.println(bc.getBalance()); // 1000.0
```

- 获取类的方法（如何查找 Method）

- 常用方法：

- Method getMethod(name, Class...): 按名和参数类型获取公共方法（包含继承的 public）
- Method getDeclaredMethod(name, Class...): 按名和参数类型获取当前类声明的方法（不包含继承的方法）
- Method[] getMethods(): 获取所有公共方法（包含继承来的）
- Method[] getDeclaredMethods(): 获取当前类声明的所有方法（包含 public/protected/default/private，但不包含继承的方法）

- 参数说明：查找带参数的方法时需要传入参数类型的 Class 对象数组（例如 int.class）

- Method 类提供方法级的元信息与操作

- Method 对象可以提供方法名、返回类型、参数类型、注解、参数数量等信息：

- getName() → 方法名
- getReturnType() → 返回类型（返回 Class<?>）
- getParameterTypes() / getParameterCount() → 参数类型数组 / 参数个数
- getParameterAnnotations() → 参数注解
- getModifiers() → 用整数表示的方法修饰符，需用 Modifier 类解码（如 Modifier.isPublic(...)）

- Method 可用于在运行时读取方法签名、检查访问修饰以及准备调用

- 注意：对私有方法同样可以通过 setAccessible(true) 打开访问（受安全限制）

- 通过反射调用方法

```
String s = "Hello Java";
Method m = String.class.getMethod("substring", int.class);
System.out.println(m.invoke(s, 6)); ➔ Java
```

- 反射可以在运行时获取某个方法的描述，然后在指定对象上调用该方法并传入参数。
- 调用过程会执行目标方法的实际逻辑，相当于在运行时“动态调用”该方法。
- 反射调用可能会出现访问异常、参数不匹配异常或目标方法内部抛出的异常（需要捕获并处理）。
- 这里的 null 就是因为要调用的是一个静态方法（static method）

```
Method m2 = Integer.class.getMethod("parseInt", String.class);
System.out.println(m2.invoke(null, "12345").getClass().getName());
```

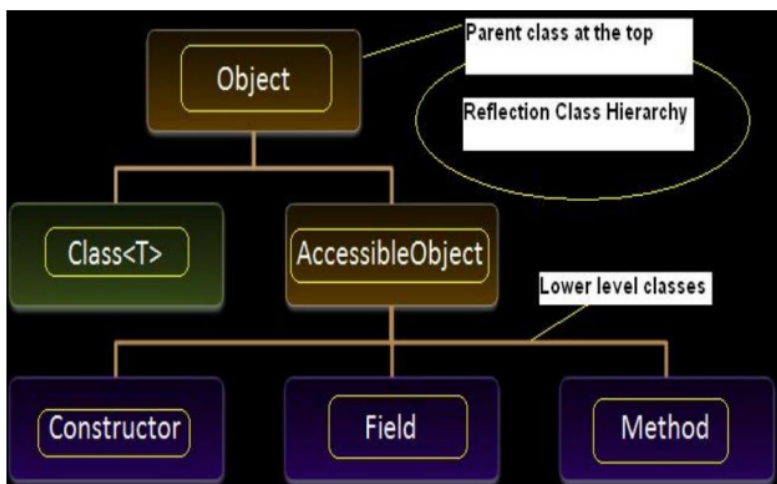
Why null?

- Reflection 中的方法调用签名是：
 - Object Method.invoke(Object obj, Object... args)
 - 第一个参数 obj 表示要在其上调用该方法的实例（即目标对象）。
- 对于实例方法，你必须传递该实例；但对于静态方法，方法并不属于某个实例，obj 参数会被忽略，所以可以传 null。
- Integer.parseInt(String) 是一个 public static 方法，所以用 m2.invoke(null, "12345") 是合法且常见的做法。
- 注意返回值类型：parseInt 返回的是原始类型 int，但 invoke 会把原始返回值自动装箱（boxing）成 Integer 并作为 Object 返回。因此 .getClass().getName() 会输出 "java.lang.Integer"。

```
Method m2 = Integer.class.getMethod("parseInt", String.class);
Object ret = m2.invoke(null, "12345");           // 第一个参数为 null 因为 parseInt 是 static
System.out.println(ret.getClass().getName());    // 输出: java.lang.Integer
System.out.println(ret);                        // 输出: 12345
```

- 静态方法属于类本身而不是某个实例，因此通过反射调用静态方法时，传给调用者的“目标对象”参数可以为 null。
- 通过反射得到的返回值仍是正常的对象，可以进一步通过反射或类型检查获得其类名或其它信息。
- 注意：对静态方法传入非 null 也不会把该方法变为实例方法，但通常使用 null 更能表达该方法的静态语义。
- 方法的可访问性与访问检查：
 - 默认情况下，反射在调用方法时会遵守 JVM 的访问控制规则（例如 private、protected、包访问限制）。
 - 如果从类外部调用私有方法或不满足可见性规则的受保护方法，会抛出访问相关的异常。

- 可以通过在反射获得的方法对象上设置“可访问”标志来临时关闭访问检查，从而绕过可见性限制（前提是运行环境允许）。
- 关闭访问检查要谨慎使用，可能受安全管理器或模块系统限制，并且会破坏封装性，仅在可信或受控场景（如测试、框架实现）中使用。
- AccessibleObject 是反射可访问对象的基类



- AccessibleObject 是 Field、Method 和 Constructor 等反射对象的共同父类，负责管理访问控制相关功能。
- 通过它可以检查某个反射对象在当前上下文是否可访问，也可以设置或尝试设置其“可访问”状态以抑制访问检查。
- 常见用途包括：批量修改可访问性、在受控环境下允许访问私有成员、先检查再调用以避免异常。
- 同样需注意安全与稳定性：修改可访问性可能被安全策略或运行时机制拒绝，应在合适场景下使用。
- 使用反射创建对象（通过构造器）
 - 可以使用 `getConstructor(Class<?>...)` 获取指定的公共构造器（按参数类型定位）。
 - `getConstructor` 返回一个 `Constructor` 对象，代表该类的指定公共构造器。
 - 通过 `Constructor.newInstance(...)` 使用该构造器创建实例（传入对应的参数）。
 - 示例流程：先由 `Class` 得到 `Constructor`，再调用 `newInstance("Alice", 15)` 创建 `Student` 实例。

```
Class clz = Student.class;
Constructor constructor = clz.getConstructor(String.class, int.class);
Student std = (Student) constructor.newInstance("Alice", 15);
```

- 注意：若要访问非公共构造器可用 `getDeclaredConstructor` 并配合 `setAccessible(true)`，但受安全限制且需谨慎。
-


```

Class ac = ArrayList.class; 获取直接父类
Class sc = ac.getSuperclass();
System.out.println(sc);
System.out.println("-----");
Class[] ai = ac.getInterfaces();
for (Class i : ai) { 获取该类直接实现的接口列表
    System.out.println(i);
}

```

```

class java.util.AbstractList
-----
interface java.util.List
interface java.util.RandomAccess
interface java.lang.Cloneable
interface java.io.Serializable

```

- 反射的实际应用场景：
 - 单元测试（JUnit）：
 - 测试框架常用反射遍历测试类方法，自动发现以特定命名或注解标记的方法并作为测试用例运行。
 - 在测试中有时需要设置私有字段或调用私有方法以便构造测试场景。
 - 框架（以 Spring 为例）：
 - 依赖注入（DI）大量使用反射来识别依赖（例如构造器参数类型）并在运行时创建实例。
 - 运行时生成或注入对象实例到其他对象中，实现解耦与自动装配。
 - 其它场景：序列化/反序列化、ORM 映射、动态代理、插件/扩展机制等。
- 反射的缺点与风险：
 - 风险：可以设置私有或 final 字段、调用私有方法，可能破坏封装性与不变量。
 - 失去编译期类型安全：编译器无法验证反射调用是否正确，错误会在运行时暴露，调试与定位更困难。
 - 重构时易引入 bug：若按字符串名称访问成员，重命名成员不会被大多数重构工具自动更新，容易产生难以发现的问题。
 - 性能较差：反射调用在运行时开销较大，频繁大量使用可能成为性能瓶颈；应在必要时谨慎使用或做缓存优化。

• Annotation

- Java 注解概览
 - 注解以 @ 开头，是附加在类、接口、字段、方法等程序实体上的元数据（关于数据的数据）。
 - 注解不会改变已编译程序的语义（即注解本身不直接改变程序的执行行为）。
 - 注解不能被简单地当作注释处理，因为它们可能会改变编译器或运行时对程序的处理方式（例如被工具或框架读取并产生影响）。
- 注解通常用于提供额外信息（按时机分类）
 - 编译期指令：编译器可使用注解来发现问题或抑制警告（如 @SuppressWarnings）。
 - 构建期指令：构建工具可扫描注解并基于注解生成源代码或其它文件（例如生成 XML、配置信息等）。

- 运行期指令：部分注解在运行时可通过反射读取，用于框架或运行时行为的控制（例如依赖注入、序列化规则等）。
- 注解的类别

- 预定义注解 built-in annotation: @Override

- 包括语言或库中定义的内置注解（例如 @Override 等）和用于注解其它注解的元注解。

- 自定义注解 custom annotation: @interface

- 用户或框架自行定义的注解类型，用于传递特定语义或被工具/框架处理。
 - 注解类型隐式地扩展标记接口 java.lang.annotation.Annotation。
 - 使用关键字 @interface 来声明新的注解类型。
 - 注解类型的声明形式与普通接口类似（声明注解元素而不是方法实现）。
 - 用注解替代类头注释的示例（定义注解类型）

```
public class Generation3List extends Generation2List {  
    // Author: John Doe  
    // Date: 3/17/2002  
    // Current revision: 6  
    // Last modified: 4/12/2004  
    // By: Jane Doe  
    // Reviewers: Alice, Bill, Cindy  
  
    // class code goes here  
}  
  
@ClassPreamble (  
    author = "John Doe",  
    date = "3/17/2002",  
    currentRevision = 6,  
    lastModified = "4/12/2004",  
    lastModifiedBy = "Jane Doe",  
    // Note array notation  
    reviewers = {"Alice", "Bob", "Cindy"}  
)  
public class Generation3List extends Generation2List {  
  
    // class code goes here  
}
```

- 许多团队会在类体开头写注释（作者、日期、版本、审阅者等）；这些信息可以用注解来替代。
 - 定义注解类型时可以声明若干“元素”（看起来像无实现的方法），例如 String author(); String date();。
 - 注解元素支持默认值：例如 int currentRevision() default 1; String lastModified() default "N/A";。
 - 注解元素可以返回数组类型（如 String[] reviewers();）以表示多值。

- 元注解 meta-annotation:

- 元注解是作用于其它注解类型的注解，定义注解本身的语义或约束。
 - Java 在 java.lang.annotation 包中定义了若干常用元注解，例如：
 - @Documented（使注解出现在文档中）
 - 作用：标注某注解类型应默认出现在 Javadoc 文档中。
 - 含义：带有此元注解的自定义注解，当生成 Javadoc 时，会将该注解及其属性信息包含在文档里，便于用户查看注解用途与参数。
 - 使用场景：用于那些对 API 使用者有说明价值、应当出现在 API 文档中的注解（例如类级别的描述性注解）。


```

@Documented
@interface ClassPreamble {

    // Annotation element definitions

}

```

The `@ClassPreamble` now is available in the Javadoc of `Generation3List`

```

@classPreamble (
    author = "John Doe",
    date = "3/17/2002",
    currentRevision = 6,
    lastModified = "4/12/2004",
    lastModifiedBy = "Jane Doe",
    // Note array notation
    reviewers = {"Alice", "Bob", "Cindy"}
)
public class Generation3List extends Generation2List {

    // class code goes here

}

```

- `@Target`（限制注解的适用目标）

- 作用：标注某注解类型可以应用于哪些 Java 程序元素（例如类、方法、字段等）。
- 常见的 `ElementType` 示例：
 - `ANNOTATION_TYPE`：可用于注解类型本身（元注解）。
 - `CONSTRUCTOR`：可用于构造器。
 - `FIELD`：可用于字段或属性。
 - `LOCAL_VARIABLE`：可用于局部变量。
 - `METHOD`：可用于方法。
 - `PACKAGE`：可用于包声明。
 - `PARAMETER`：可用于方法参数。
 - `TYPE`：可用于类、接口、枚举或注解类型等任何类型元素。
- 作用效果：通过 `@Target` 可以在编译期限制注解的误用，提高注解使用的准确性。

-

```

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.SOURCE)
public @interface Override {
}

```

```

@Documented
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface FunctionalInterface {}

```

```

@Documented
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.CONSTRUCTOR, ElementType.METHOD})
public @interface SafeVarargs {}

```

- 示例：常见注解会配合 `@Target` 指定目标（例如 `@Override` 针对方法，`@SafeVarargs` 可指定方法或构造器）。
- 建议：为自定义注解明确指定 `@Target`，避免注解被用在不适当的位置，从而让编译器及工具更好地校验注解用法。
- 补充：`@Target` 通常与 `@Retention`（注解的保留策略）一起使用，以共同定义注解的可见范围和适用位置。

- `@Retention`（注解保留策略）

- 说明：用来指定注解在什么阶段可用（注解被保存到哪里）。
- 常用三种保留策略：
 - `RetentionPolicy.SOURCE`：注解仅保留在源码中，编译器会忽略（不会出现在 `.class` 文件中）。

- `RetentionPolicy.CLASS`：注解在编译时保留并写入 `.class` 文件，但运行时 JVM 会忽略（类加载时不会保留给运行时访问）。
- `RetentionPolicy.RUNTIME`：注解在运行时可用，JVM 会保留注解信息，可通过反射读取。
- 选择建议：根据注解的用途（仅供编译期检查、供字节码工具处理或需在运行时读取）来设置合适的 `RetentionPolicy`。
- 标注的注解只保存在源代码中；编译后不写入 `.class` 文件，运行时不可见。

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.SOURCE)
public @interface Override {
}
```

```
@Target({TYPE, FIELD, METHOD, PARAMETER,
@Retention(RetentionPolicy.SOURCE)
public @interface SuppressWarnings {
```

- `RetentionPolicy.CLASS`（写入字节码但运行时不可见）
 - 特点：注解会被编译器保留并写入 `.class` 文件，但类加载后默认不会被 JVM 在运行时提供访问（不是运行时可读）。
 - 默认策略：这是注解的默认保留策略（若未指定 `Retention` 则通常为 `CLASS`）。
 - 适用场景：适合供字节码工具、静态分析器或混淆器等在处理 `.class` 文件时读取的注解，而不需要在运行时通过反射访问。

```
@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Range {
    int min() default 0;
    int max() default 255;
}
```

RUNTIME policy signals to the Java compiler and JVM that the annotation should be available via reflection at runtime.

Example adapted from <https://www.liaoxuefeng.com/wiki/1252599548343744/1265102026065728>

```
public class Person {
    @Range(min=3, max=20)
    public String name;

    @Range(max=10)
    public String city;

    @Range(min=1, max=100)
    public int age;

    public Person(String name, String city, int age) {
        this.name = name;
        this.city = city;
        this.age = age;
    }
}
```

- `@ Target(ElementType.FIELD)`：该注解只能用于字段上。
- `@ Retention(RetentionPolicy.RUNTIME)`：注解在编译后会保留到运行时，JVM 与反射机制可以在运行时读取到它。
- 用反射读取注解并校验的框架代码

RetentionPolicy.RUNTIME

Since @Range is available at runtime, we can check it using reflection.

If we use SOURCE or CLASS retention policy for @Range, it will not be available at runtime (range == null in the code)

```
public static void check(Person person) throws IllegalArgumentException {
    // go through each field
    for(Field field: person.getClass().getFields()){
        // get the @Range annotation of the field
        Range range = field.getAnnotation(Range.class);
        // if there is a @Range annotation
        if (range != null){
            // get the value of the field
            Object value = field.get(person);
            if (value instanceof String){
                String s = (String) value;
                if (s.length() < range.min() || s.length() > range.max()){
                    throw new IllegalArgumentException("Invalid range of " +
                        "of string field: " + field.getName());
                }
            }
            else{
                int i = (int) value;
                if (i < range.min() || i > range.max()){
                    throw new IllegalArgumentException("Invalid range of " +
                        "int field: " + field.getName());
                }
            }
        }
    }
}
```

- 展示了一段框架/工具函数的代码，它用反射遍历 Person 的字段，读取 @Range 注解、取得字段值并做校验（字符串长度或整数范围），不满足则抛出异常。

```
Person p1 = new Person( name: "Alice", city: "Beijing", age: 20);
Person p2 = new Person( name: "a", city: "Beijing", age: 20);
Person p3 = new Person( name: "Alice", city: "The city name is Beijing", age: 20);
Person p4 = new Person( name: "Alice", city: "Shenzhen", age: 200);

check(p1); OK
check(p2); java.lang.IllegalArgumentException: Invalid range of string field: name
check(p3); java.lang.IllegalArgumentException: Invalid range of string field: city
check(p4); java.lang.IllegalArgumentException: Invalid range of int field: age
```

- @ Inherited
- @ Repeatable

- 使用元注解可以控制注解是否出现在 Javadoc、注解可用的位置、注解的保留期、是否可继承或是否可重复使用等行为。

- Java 平台中常见的注解类型示例（定义在 java.lang 或相关包中）：

- @Deprecated（标记弃用）

- 作用：标记某个元素（类、方法、字段等）已被弃用，不应再使用。

```
Date dt = new Date( year: 2002, month: 12, date: 20);

'Date(int, int, int)' is deprecated

@Deprecated
@Contract(pure = true)
public Date(
    int year,
    @MagicConstant(intValues = {Cal
    int date
})
```

- 编译器行为：当程序使用被 @Deprecated 标注的元素时，编译器会产生警告，提醒开发者该元素不再推荐使用。
- 使用场景：提醒替代 API、逐步废弃旧功能、提示升级或替换方案。
- 注意：标注本身不改变程序语义，只是作为元信息由编译器或工具检测并提示。

- @Override（标注覆盖父类方法）

- 作用：告诉编译器该方法是用来覆盖（重写）父类中声明的方法。
- 好处：作为意图的文档说明，并让编译器帮忙检查是否确实正确覆盖（参数类型、方法签名、可见性等）。

- 防错：若标注的方法未正确覆盖父类方法（例如方法名或参数写错），编译器会报错，帮助早期发现问题。
- 建议：重写父类方法时应尽量使用 `@Override`，以提高代码可靠性和可维护性。
- 未使用 `@Override` 导致的问题

```
import java.awt.*;
import java.awt.event.*;
public class AnnotationOverrideDemo extends Frame {
    public AnnotationOverrideDemo() {
        this.addWindowListener(new WindowAdapter() {
            public void windowclosing(WindowEvent e) {
                System.exit(0);
            }
        });
        setSize(200, 100);
        setTitle("Annotation Override Demo");
        setVisible(true);
    }
    public static void main(String[] args) { new AnnotationOverrideDemo(); }
}
```

- 问题描述：代码编译并运行，但由于方法名/签名写错（例如写成 `windowclosing` 而非 `windowClosing`），期望的回调不会被触发（例如窗口关闭按钮无效）。
 - 原因：没有正确覆盖父类或接口的方法，结果只是定义了一个新的方法而非重写。
 - 解决：在方法上添加 `@Override`，编译器会提示“未覆盖父类方法”的错误，从而暴露拼写或签名错误。
 - 建议：使用 `@Override` 作为防错和自文档化手段，尤其在实现监听器或回调时非常有用。
- 添加 `@Override` 后捕获错误

```
@Override
public void windowclosing(WindowEvent e) {
    System.exit(0);
}
```

Should be windowClosing

```
@Override
public void windowclosing(WindowEvent e) {
    System.exit(0);
}
```

Method does not override method from its superclass

- 操作：在错误命名的方法上添加 `@Override`（意图重写），编译器会检测并报错，指出方法没有覆盖其父类方法。
 - 效果：通过编译错误可以及时发现应为 `windowClosing` 的拼写/签名问题并修正，避免运行时功能失效。
 - 小结：`@Override` 不仅作标注，也用作编译期校验工具，能有效减少由于拼写或签名错误导致的逻辑缺陷。
- `@SuppressWarnings`（抑制编译器警告）
 - 作用：告诉编译器抑制某些编译时会产生的特定警告。
 - 警告分类：Java 规范将编译器警告分为若干类别，常见的有弃用（`deprecation`）和未检查（`unchecked`）等。
 - 用法示例：同时抑制多类警告：`@SuppressWarnings({"unchecked","deprecation"})`。
 - 注意事项：应尽量局部、谨慎地使用，仅在确认该用法安全且无法消除警告时使用，避免隐藏真实问题。

```

public class SuppressWarningsTest {
    public static void addSth(List list){
        list.add("Test");
    }
    public static void main(String[] args) {
        List list = new ArrayList<>();
    }
}

```

Unchecked call to 'add(E)' as a member of raw type 'java.util.List'
Try to generify 'SuppressWarningsTest.java' Alt+Shift+Enter

```

public class SuppressWarningsTest {
    @SuppressWarnings("unchecked")
    public static void addSth(List list){
        list.add("Test");
    }
    public static void main(String[] args) {
        List list = new ArrayList<>();
    }
}

```

- 问题场景：使用原始类型（raw types）或与泛型交互时，编译器会发出 unchecked 警告。
- 解决方式：可以在产生警告的方法或局部变量上加 `@SuppressWarnings("unchecked")` 来抑制该警告。
- 最佳实践：优先修正导致警告的代码（使用泛型等），仅在确实安全且无法修复时局部抑制警告；避免在类或包级别滥用，防止掩盖其他潜在问题。

- @FunctionalInterface（标注函数式接口）

- @SafeVarargs（标注可安全使用可变参数的场合）

- 背景：varargs（可变参数）方法在与泛型结合时，存在堆污染（heap pollution）风险，编译器会发出警告。
- 作用：当你确信在可变参数中不会发生不安全的操作时，可用 `@SafeVarargs` 标注以抑制该警告。
- 适用范围：通常用于不会被重写的方法（例如 static 方法、final 方法或构造器等不可被覆盖的情况），以避免子类造成安全问题。
- 风险提示：只有在完全确认不存在堆污染风险时才使用，否则可能隐藏导致运行时异常的隐患。
- 在泛型可变参数方法上添加 `@SafeVarargs`，以在确定安全的前提下抑制可能的编译警告。

```

public class SafeVarargsDemo {
    public static void main(String[] args) {
        display( ...array: "10", 20, 30.00);
    }
}

```

```

public static <T> void display(T... array){
    for(T arg: array){
        System.out.print
    }
}

```

Possible heap pollution from parameterized
Annotate as '@SafeVarargs' Alt+Shift+Enter

```

public static <T> void display(
    @NotNull T... array

```

- 在方法内部不要把可变参数数组当作可写的泛型容器（避免向数组写入不同泛型类型），以防堆污染。
- 仅当你能证明方法的实现对传入数组没有不安全操作时才添加注解。
- 若方法可能被子类覆盖或会执行不安全操作，则不应使用该注解。